

A PROJECT REPORT ON LOAD BALANCER WITH HEALTH CHECKS

Submitted By

Sonali Patel – 2315002185

Tushant Pratap Singh– 2315002319

Sumit Kaushik - 2315002231

Uday Dixit - 2315002333

Udit Raghav - 2315002338

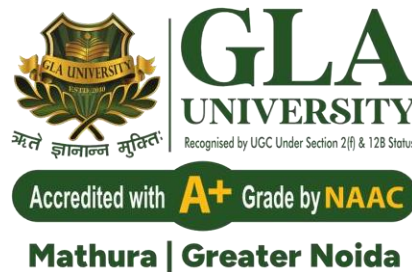
Under the Supervision Of

Mr. Ayush Kumar

GLA University, Mathura

Department of computer science & Engineering

GLA UNIVERSITY, MATHURA



GLA University, Mathura - 281406

DECLARATION

I/we hereby declare that the work which is being presented in the Bachelor of Technology project titled “**Load Balancer with Health Checks**”, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, and submitted to the Department of Computer Engineering and Applications of **GLA University, Mathura**, is an authentic record of my/our own work carried out under the supervision of **Mr. Ayush Kumar, IntegraMinds Trainer, Dept. of CEA, GLA University**.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign: _____
Name of Candidate: Sonali Patel
University Roll No.: 2315002185

Sign: _____
Name of Candidate: Sumit Kaushik
University Roll No.: 2315002231

Sign: _____
Name of Candidate: Uday Dixit
University Roll No.: 2315002333

Sign: _____
Name of Candidate: Udit Raghav
University Roll No.: 2315002338

Sign: _____
Name of Candidate: Tushant Pratap Singh
University Roll No.: 2315002319

CERTIFICATE

This is to certify that the project entitled “**Load Balancer with Health Checks**”, carried out in Mini Project-II, is a bonafide work by **Sonali Patel, Sumit Kaushik, Tushant Pratap Singh, Uday Dixit and Udit Raghav** and is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (Computer Science & Engineering)**.

Signature of Supervisor: _____

Name of Supervisor: Mr. Ayush Kumar

Date: _____

ACKNOWLEDGMENT

It is a genuine pleasure to express my profound gratitude and deep regard to my guide **Mr. Ayush Kumar** for his valuable guidance, continuous support, and constant encouragement throughout the completion of this project.

I would also like to express my sincere thanks to the faculty and staff of **GLA University, Mathura**, for providing me with the opportunity to work on this project titled “**Load Balancer with Health Checks.**” This project has helped me gain practical knowledge, strengthen my technical skills, and understand the real-world applications of containerization and full-stack development.

I am also thankful to all my teammates for their cooperation and support, which played an important role in the successful completion of this project.

INDEX	PAGE No.
1. Abstract	01-02
2.Introduction	03-04
3.System Analysis	05-07
4.Objectives and Scope of the Project	08-09
5.Functional Requirements	10-11
6.Use Case Description and Use Case Diagram	12-13
7.System Flow and Interface Design	14-16
8.Hardware & Software Requirements & Future Work	17
9.Conclusion	18
10.Refereneces / Bibliography	19

ABSTRACT

The project titled “**Load Balancer with Health Checks**” is designed to efficiently distribute incoming network traffic across multiple servers to ensure high availability, reliability, and optimal resource utilization. In modern web applications, handling large volumes of user requests is critical, and a load balancer plays a key role in preventing server overload and improving system performance.

The system works by receiving client requests and distributing them among multiple backend servers using load balancing algorithms such as Round Robin or Least Connections. This ensures that no single server is overwhelmed, resulting in faster response times and improved fault tolerance.

Key features of the system include:

- Improved system performance and scalability
- High availability and fault tolerance
- Reduced server overload
- Better resource utilization

The project is developed using modern technologies and networking concepts, ensuring reliability and scalability. It can be applied in web servers, cloud platforms, and large-scale distributed systems.

Overall, this project demonstrates how load balancing enhances system efficiency, maintains service continuity, and improves user experience in high-traffic environments.

INTRODUCTION

In today's digital era, web applications handle a large number of user requests simultaneously. Managing this traffic efficiently is crucial for maintaining system performance, reducing response time, and ensuring high availability. However, traditional systems that rely on a single server often face issues such as server overload, slow performance, and system downtime.

The **Load Balancer with Health Checks** is developed to overcome these limitations by distributing incoming network traffic across multiple servers. The system receives client requests and forwards them to backend servers efficiently, ensuring balanced workload and continuous service availability.

The project consists of three main components:

1. Client (User Requests) – Generates requests to access the application
2. Load Balancer (Node.js / Server Logic) – Distributes incoming traffic among servers
3. Backend Servers – Process requests and send responses back to the client

The system provides important functionalities such as:

- Efficient traffic distribution
- Improved response time
- High availability
- Fault tolerance
- Better resource utilization

This system helps organizations handle large-scale user traffic efficiently and maintain smooth system performance, ensuring reliable and fast service for users

SYSTEM ANALYSIS

The System Analysis phase focuses on studying traditional server management and traffic distribution systems and designing a modern load balancing solution with health monitoring. It identifies the limitations of static or manual server handling and proposes an efficient, scalable, and fault-tolerant system using modern backend technologies.

The proposed system ensures efficient distribution of incoming requests across multiple servers while continuously monitoring server health. This helps maintain high availability, reliability, and performance of applications.

A. Existing System

In traditional systems, incoming traffic is often handled by a single server or distributed manually without intelligent routing. These systems lack automated monitoring and failover mechanisms, which can lead to downtime and poor performance.

Server health is not checked continuously, and failures are often detected only after users experience issues. Load distribution is either static or inefficient, leading to uneven server utilization.

Developers and organizations face several challenges:

- **No Load Balancing** – Requests are handled by a single server, causing overload
- **Server Downtime Issues** – No automatic failover when a server crashes
- **Manual Monitoring** – Health of servers is checked manually
- **Uneven Traffic Distribution** – Some servers are overloaded while others remain idle
- **Poor Scalability** – Difficult to handle increasing traffic efficiently

- **High Response Time** – Slower performance due to overloaded servers
- **Lack of Fault Tolerance** – System fails if the main server goes down

Additionally, traditional systems do not include automated health checks, making it difficult to ensure that only active and healthy servers handle request

B. Proposed System

The proposed **Load Balancer with Health Check System** provides an intelligent and automated way to distribute incoming traffic across multiple backend servers while continuously monitoring their health.

The system receives client requests and forwards them to available servers using a load balancing algorithm. It periodically checks the health of each server and removes unhealthy servers from the pool, ensuring uninterrupted service.

System Components

- **Load Balancer Server (Node.js + Express.js)**

Handles incoming requests and distributes them across multiple backend servers

- **Backend Servers**

Multiple servers that process client requests

- **Health Check Module**

Continuously monitors server status (alive/down)

- **Routing Algorithm**

Distributes traffic efficiently (e.g., Round Robin or similar approach)

- **Monitoring System**

Tracks server availability and performance

Working of the System

1. Client sends a request to the load balancer
2. Load balancer selects an active server
3. Request is forwarded to the selected server
4. Server processes the request and sends a response
5. Health check system continuously monitors all servers
6. If a server fails, it is removed from the active pool
7. Traffic is redirected only to healthy servers

System Capabilities

Users and developers can monitor:

- Active and inactive servers
- Server response status
- Request distribution
- System uptime
- Load handling efficiency

C. Advantages

The proposed system offers several advantages over traditional systems:

- **Efficient Load Distribution** – Balances traffic across multiple servers
- **High Availability** – Ensures continuous service even if a server fails

- **Automatic Health Checks** – Detects and removes unhealthy servers instantly
- **Improved Performance** – Reduces response time by avoiding overload
- **Scalability** – Easily handles increasing traffic by adding more servers
- **Fault Tolerance** – System continues working even during failures
- **Reduced Downtime** – Minimizes service interruptions
- **Better Resource Utilization** – Ensures all servers are used effectively

OBJECTIVE AND SCOPE OF THE PROJECT

Objectives

The main objective of this project is to design and develop an efficient **Load Balancer with Health Check System** using modern backend technologies. The system aims to distribute incoming network traffic across multiple servers while continuously monitoring their health to ensure high availability, reliability, and optimal performance.

The key objectives include:

- To design and develop a load balancing system using Node.js and Express.js
- To distribute incoming client requests efficiently across multiple backend servers
- To implement a load balancing algorithm such as Round Robin for fair traffic distribution
- To continuously monitor the health status of all backend servers
- To automatically detect and remove unhealthy or non-responsive servers
- To ensure high availability and fault tolerance in the system
- To minimize server overload and improve response time
- To build a system that can handle increasing traffic efficiently
- To provide a reliable and scalable backend infrastructure
- To improve overall system performance and uptime

Scope

The scope of the project defines the functionalities and boundaries of the **Load Balancer with Health Check System**. The system is designed to help developers and organizations efficiently manage server traffic and ensure continuous service availability.

- The system provides an automated load balancing mechanism to distribute requests across multiple servers
- It continuously monitors server health using periodic health checks
- The system automatically excludes failed or unhealthy servers from the request pool
- It ensures that only active and healthy servers handle client requests
- The platform improves performance by reducing server overload and response time
- It maintains system stability even when one or more servers fail
- The system is scalable and can support additional servers as traffic increases
- It operates as a backend service and can be integrated with web applications or APIs
- The system can handle concurrent user requests efficiently
- It provides a foundation for future enhancements such as advanced load balancing algorithms, logging, monitoring dashboards, and cloud deployment

FUNCTIONAL REQUIREMENTS

- **Request Handling**

The system receives incoming client requests and processes them through a centralized load balancer server built using Node.js and Express.js.

- **Load Distribution**

The system distributes incoming requests across multiple backend servers using a load balancing algorithm such as Round Robin to ensure fair and efficient utilization of resources.

- **Server Health Monitoring**

The system continuously monitors the health status of all backend servers using periodic health check mechanisms to determine whether servers are active or inactive.

- **Automatic Failover**

If a server becomes unresponsive or fails, the system automatically removes it from the active server pool and redirects traffic to healthy servers.

- **Dynamic Server Management**

The system dynamically updates the list of active servers based on their health status, ensuring only functional servers handle requests.

- **Response Handling**

The system collects responses from backend servers and sends them back to the client efficiently with minimal delay.

- **Performance Optimization**

The system ensures optimal performance by preventing server overload and reducing response time through balanced traffic distribution.

- **Logging and Monitoring**

The system maintains logs of requests, server status, and failures for monitoring and debugging purposes.

- **Error Handling**

The system handles errors such as server downtime, failed requests, and connection issues gracefully, ensuring minimal disruption to users.

- **Technical Requirement**

The system follows a client-server architecture and requires:

- Active internet connection
- Backend developed using Node.js and Express.js
- Multiple backend servers for request handling
- Health check mechanism for monitoring servers

INTERFACE REQUIREMENTS

The system includes interfaces that help developers and administrators monitor and manage load balancing operations efficiently.

1. Client Request Interface

- This represents the entry point where users or applications send requests to the load balancer
- Requests can be generated through browsers, APIs, or client applications

2. Load Balancer Interface (Core System)

- This is the main component of the system
- It receives all incoming requests and distributes them across backend servers
- It applies load balancing algorithms to ensure efficient traffic handling

3. Server Status Interface

- Displays the status of all backend servers (active/inactive)
- Helps in identifying healthy and failed servers
- Updated dynamically through health checks

4. Health Check Module Interface

- Continuously monitors server availability
- Sends periodic requests (pings) to servers
- Updates system based on server responses

5. Request Routing Section

- Shows how requests are being distributed among servers
- Ensures equal and efficient load sharing

6. Logging and Monitoring Interface

- Displays logs of incoming requests, server responses, and failures
- Helps in debugging and performance analysis

USE CASE DESCRIPTION AND USE CASE DIAGRAM

Pre-condition:

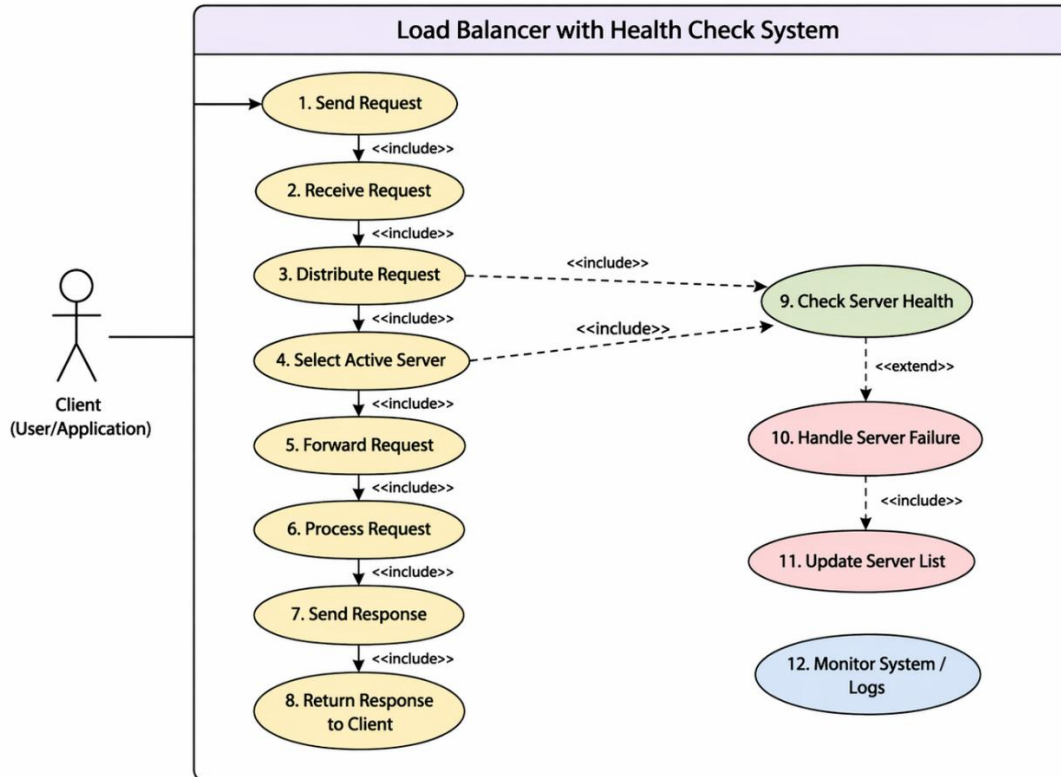
User (client/application) must send requests to the load balancer system so that traffic can be distributed across multiple servers and monitored efficiently.

Use Case Description

Use Case Name	Description
Client	A user or application that interacts with the system by sending requests
Send Request	Client sends a request to the load balancer server
Receive Request	The load balancer receives incoming client requests
Distribute Request	The load balancer distributes requests across multiple backend servers
Check Server Health	The system monitors server status (active/inactive) continuously
Select Active Server	The system selects a healthy server from the available pool
Forward Request	The load balancer forwards the request to the selected backend server
Process Request	The backend server processes the client request
Send Response	The backend server sends the response back to the load balancer
Return Response	The load balancer returns the response to the client
Handle Server Failure	The system detects server failure and removes it from active servers
Update Server List	The system updates the list of available servers dynamically
System Monitoring	The system logs and monitors request handling and server performance

User Case Diagram:

Use Case Diagram: Load Balancer with Health Check System



SYSTEM FLOW AND INTERFACE DESIGN

The system flow describes how client requests are received, processed, and distributed across multiple servers efficiently. The interface design represents different components of the system that help monitor load balancing and server health effectively.

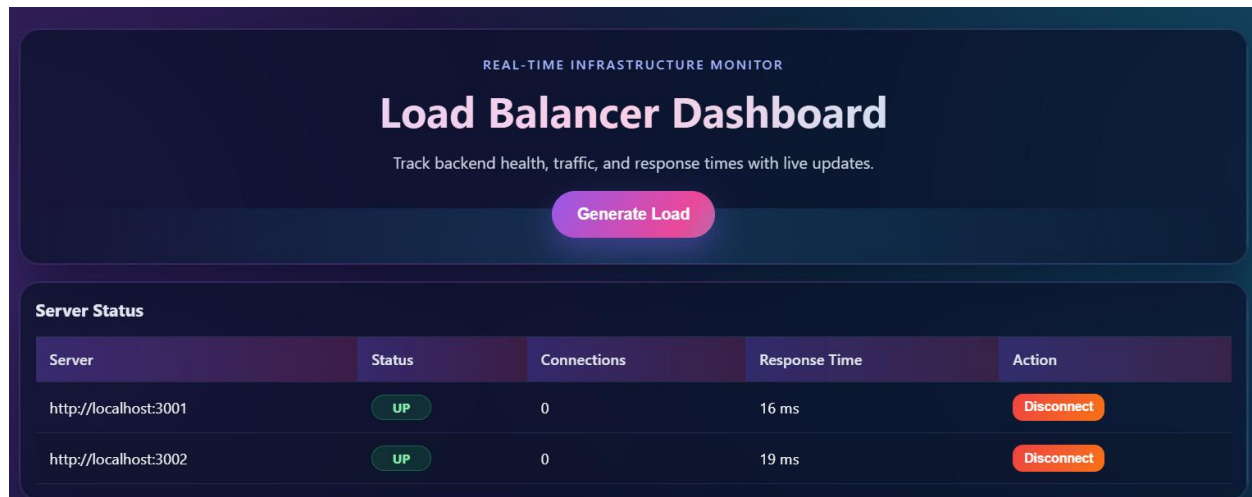
1. Client Request (Entry Point Interface)

This is the main entry point where users or applications interact with the system.

Clients perform actions such as:

- Sending API requests
- Accessing web applications
- Requesting data or services

These requests are received by the load balancer and forwarded to backend servers for processing.



2. Monitoring Dashboard

The dashboard is the core interface of the system. It displays real-time system monitoring including:

- Active Servers
- Inactive Servers
- Total Requests Processed
- Request Distribution
- Server Status

It also includes:

- Health Check Latency Graph (shows response time of servers over time)
- Server Health Status Monitoring
- Real-time Performance Tracking
- Logs and Activity Monitoring

The dashboard visually represents the Recent Health History Trend, where spikes indicate higher latency or potential server issues, helping in quick detection of performance problems.



3. Live Server Monitoring Feed

This section shows real-time server activity in tabular format. It includes:

- Server URL
- Status (UP/DOWN)
- Latency (ms)
- Checked Time

This helps in quickly monitoring server health.

It also shows backend changes like:

- Action (RECONNECT / DISCONNECT)
- Server
- Timestamp

This helps track server updates in real time.

Recent Health History			
Server	Status	Latency	Checked
http://localhost:3002	UP	52 ms	9:58:42 pm
http://localhost:3001	UP	216 ms	9:58:42 pm
http://localhost:3002	UP	6 ms	9:22:06 pm
http://localhost:3001	UP	6 ms	9:22:06 pm
http://localhost:3002	UP	6 ms	9:22:01 pm
http://localhost:3001	UP	4 ms	9:22:01 pm
http://localhost:3002	UP	3 ms	9:21:56 pm
http://localhost:3001	UP	3 ms	9:21:56 pm
http://localhost:3002	UP	5 ms	9:21:51 pm
http://localhost:3001	UP	5 ms	9:21:51 pm

Backend Config Changes			
Action	Server	Git Commit	At
RECONNECT	http://localhost:3002	f6dda92	12:17:27 pm
DISCONNECT	http://localhost:3002	f6dda92	12:17:14 pm
RECONNECT	http://localhost:3001	f6dda92	12:17:04 pm
DISCONNECT	http://localhost:3001	f6dda92	12:15:12 pm
RECONNECT	http://localhost:3002	f6dda92	12:33:08 am
RECONNECT	http://localhost:3001	f6dda92	12:33:06 am
DISCONNECT	http://localhost:3002	f6dda92	12:32:53 am
DISCONNECT	http://localhost:3001	f6dda92	12:32:44 am
RECONNECT	http://localhost:3002	f6dda92	11:53:13 pm
DISCONNECT	http://localhost:3002	f6dda92	11:53:09 pm

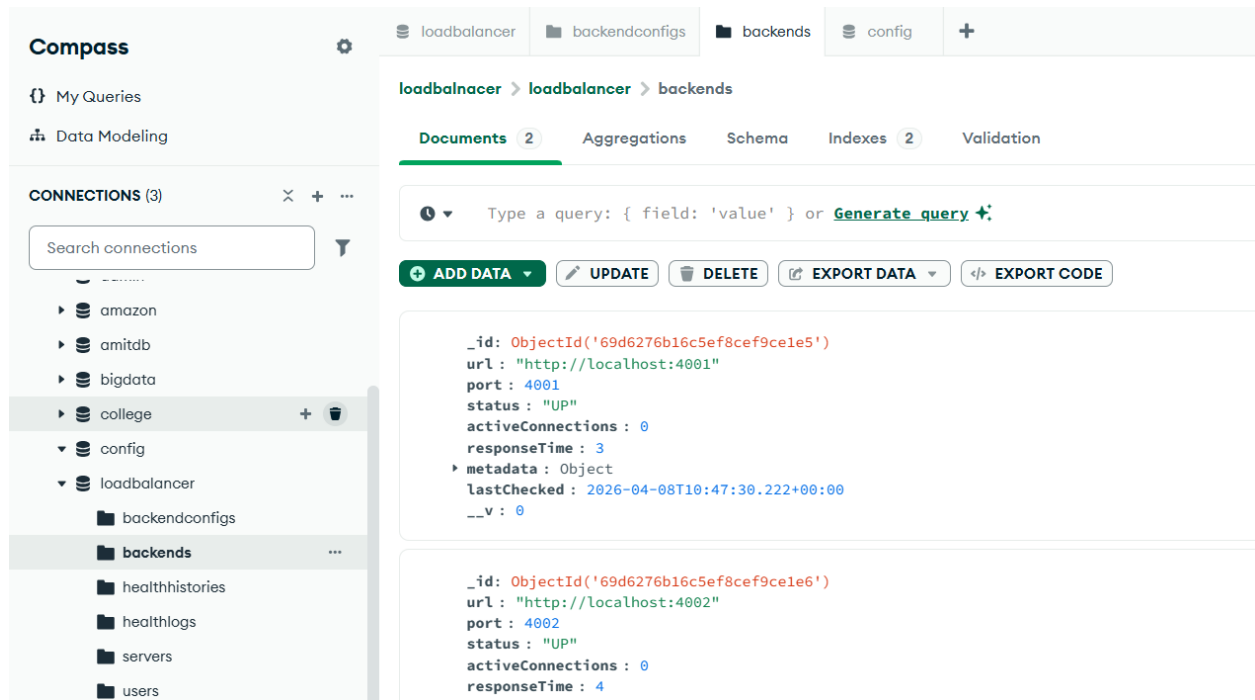
4. Database (MongoDB Interface)

This represents the backend database where all event data is stored.

Each event contains:

- User ID
- Event Type
- Timestamp
- Device Information

The database ensures secure storage and retrieval of analytics data.



HARDWARE & SOFTWARE TO BE USED

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: 4 GB or more
- Storage: 128 GB SSD or higher (recommended for better performance)
- Internet Connection: Required for real-time data processing and API communication

Software Requirements:

- Frontend: HTML, CSS, JavaScript, EJS
- Backend: Node.js, Express.js
- Database: MongoDB

Tools Used:

- Visual Studio Code (for development)
- Git & GitHub (for version control)

- Postman (for API testing)
- Browser: Google Chrome or any modern web browser

Future Work of this Project

- **Develop a Monitoring Dashboard UI**

The system can be extended with a full frontend dashboard to visually monitor server status, request distribution, and performance metrics in real time.

- **Advanced Load Balancing Algorithms**

Enhance the system by implementing advanced algorithms such as Least Connections, Weighted Round Robin, or IP Hash for better traffic distribution.

- **Auto-Scaling Support**

Integrate auto-scaling features to automatically add or remove servers based on traffic load.

- **Real-Time Alerts & Notifications**

Implement alert systems (email/SMS) to notify administrators about server failures or high latency.

- **User Authentication & Access Control**

Add secure login and role-based access (Admin/User) to manage and monitor the system securely.

- **Cloud Deployment**

Deploy the system on cloud platforms like AWS, Azure, or Google Cloud for better scalability and availability.

- **Integration with Monitoring Tools**

Integrate with tools like Prometheus or Grafana for advanced monitoring and visualization.

- **Logging & Analytics Enhancement**

Improve logging mechanisms to analyze traffic patterns, server performance, and failure trends.

- **Custom Configuration Options**

Allow users to configure server lists, health check intervals, and load balancing strategies dynamically.

CONCLUSION

The project “**Load Balancer with Health Check System**” successfully demonstrates how modern backend technologies can be used to build an efficient, reliable, and scalable system for managing and distributing network traffic. The system simplifies the process of handling multiple client requests by intelligently distributing them across available servers, eliminating the limitations of traditional single-server systems.

By integrating technologies such as **Node.js** and **Express.js**, the project ensures smooth request handling, efficient traffic distribution, and continuous server monitoring. The system performs real-time health checks to identify active and inactive servers, ensuring that only healthy servers handle incoming requests. This improves system reliability and reduces downtime significantly.

Overall, this project enhances system performance by reducing server overload, improving response time, and ensuring high availability. It helps developers and organizations maintain stable applications, efficiently utilize resources, and handle increasing traffic demands. The system is scalable and can be extended with advanced features such as intelligent load balancing algorithms, cloud deployment, and real-time monitoring dashboards.

Hence, the project achieves its primary goal of delivering a modern, fault-tolerant, and efficient load balancing solution that ensures continuous service availability and aligns with current industry standards.

REFERENCES / BIBLIOGRAPHY

1. ? W3Schools – <https://www.w3schools.com>
(For learning HTML, CSS, JavaScript, and basic web technologies)
2. ? HTML Official Documentation – <https://html.spec.whatwg.org>
(For understanding HTML structure and standards)
3. ? MDN Web Docs – <https://developer.mozilla.org>
(For detailed documentation on HTML, CSS, JavaScript, and web APIs)
4. ? CSS Official Documentation – <https://www.w3.org/Style/CSS/>
(For styling, layouts, and responsive design concepts)
5. ? GeeksforGeeks – <https://www.geeksforgeeks.org>
(For understanding backend development, system design, and data structures)
6. ? JavaScript Info – <https://javascript.info>
(For in-depth knowledge of JavaScript concepts and event handling)
7. ? Stack Overflow – <https://stackoverflow.com>
(For debugging errors and resolving technical issues during development)
8. ? FreeCodeCamp – <https://www.freecodecamp.org>
(For practical tutorials on frontend development and real-world projects)
9. ? YouTube
(For practical tutorials on HTML, CSS, JavaScript, Node.js, MongoDB, and real-time analytics concepts)
10. ? Official Documentation:
11. ? • Node.js – <https://nodejs.org/en/docs>
(For backend development and server-side programming)
12. ? • MongoDB – <https://www.mongodb.com/docs>
(For database design, queries, and data storage)
13. ? • Express.js – <https://expressjs.com>
(For building APIs and handling backend routes)